

CRICKETAI: AI-POWERED VIRTUAL CRICKET COACH USING POSE ESTIMATION AND RETRIEVAL-AUGMENTED GENERATION

Abdul Wahid H – 24623102

Sarveshwaran K – 24623153

Vignesh P – 24623503

MINI PROJECT REPORT

Submitted to the

SCHOOL OF ENGINEERING

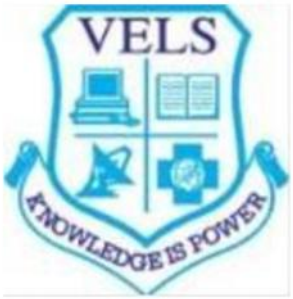
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

in Partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY CSE

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI – 600 117

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report "**CRICKETAI: AI-POWERED VIRTUAL CRICKET COACH USING POSE ESTIMATION AND RETRIEVAL-AUGMENTED GENERATION**" is the bonafide work of **ABDUL WAHID H – 24623102**, **SARVESHWARAN K – 24623153** and **VIGNESH P – 24623503**. They carried out this project under the guidance and supervision of the Department of Artificial Intelligence and Machine Learning, School of Engineering, Vels Institute of Science, Technology & Advanced Studies (VISTAS), Chennai.

The report explains the design and the working of an AI cricket coaching web application. The system uses computer vision to study a batting or bowling video, gives a technique report with scores and drills, and has an AI coach that answers questions using the player's own report and cricket coaching manuals.

SIGNATURE OF THE HOD

Dr. K. Ulaga Priya M.E., Ph.D
Associate Professor & Head
of Department AI&ML
School of Engineering
VISTAS, Chennai.

SIGNATURE OF THE SUPERVISOR

DR N MEENAKSHI M.Tech., Ph.D
Assistant Professor
Department of AI&ML
School of Engineering
VISTAS, Chennai.

Submitted for the Project Work and Viva-Voce examination held on _____ at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We thank Vels Institute of Science, Technology & Advanced Studies (VISTAS), Chennai, for giving us a good place to study and the facilities we needed to do this project.

We are very thankful to **Dr. K. Ulaga Priya M.E., Ph.D**, Associate Professor and **Head of the Department** of Artificial Intelligence and Machine Learning, for her support and encouragement during the project.

We sincerely thank our project supervisor, **DR N MEENAKSHI M.Tech., Ph.D** Assistant Professor, Department of Artificial Intelligence and Machine Learning, for her guidance and useful suggestions. Her help made it easy for us to shape the project and to write this report.

We also thank our faculty members, friends and everyone who helped us with the development, the video recording at the ground, the app testing and the project demo.

Last of all, we thank our families for their love and support all through this work.

ABDUL WAHID H - 24623102
SARVESHWARAN K - 24623153
VIGNESH P - 24623503

CHAPTER NO.	TABLE OF CONTENT	PAGE NO.
	ABSTRACT	i
	LIST OF FIGURES	ii
	ABBREVIATIONS	iii
1	**INTRODUCTION**	1
	1.1 Overview of the Project	1
	1.2 Problem Statement	1
	1.3 Objectives	1
	1.4 Scope of the Project	2
	1.5 Applications and Expected Outcomes	2
2	**LITERATURE REVIEW**	3
	2.1 Background	3
	2.2 Existing Approaches	3
	2.3 Research Gap	3
	2.4 Proposed Contribution	4
3	**SYSTEM DESIGN**	5
	3.1 System Architecture	5
	3.1.1 Overall Data Flow	6
	3.2 Hardware and Software Requirements	7
	3.3 Software Architecture	7
	3.4 Data and Analysis Flow	8
	3.5 AI Coach Design	8
4	**IMPLEMENTATION**	10
	4.1 Landing Page and Session Upload	10
	4.2 Technique Report	11
	4.3 Practice Planner and Skills Library	13
	4.4 AI Coach Workflow	14
	4.4.1 Step-by-Step Coaching Session	15
	4.5 Core Logic Summary	16
	4.6 Testing and Observations	17
	4.7 Source Code Implementation	18
	4.8 Results and Discussion	23
5	**CONCLUSION AND FUTURE WORK**	25
	5.1 Conclusion	25
	5.2 Limitations	25
	5.3 Future Enhancements	25
	5.4 Final Remark	27
	APPENDIX A - CONFERENCE PAPER CERTIFICATES	28
	REFERENCES	29

ABSTRACT

Many young cricket players practise without a coach watching them. A good coach is costly and is not always available, and a phone video of a practice session is hard to understand without expert help. CRICKETAI is a web application made to solve this problem. It works as a virtual cricket coach that studies the player's own video and explains what to improve.

In our system, the player chooses batting or bowling, gives the bowling arm and style when needed, and uploads one short clip. The server reads the video with OpenCV and finds 33 body points in every frame with MediaPipe Pose. Each joint area is scored from 0 to 100 and grouped by body part. The report shows a technique score, a tracking confidence value, strengths, areas to improve, a coaching note and corrective drills. In our sample spin bowling clip, the report gave a score of 90 out of 100 with 93% tracking confidence.

The project also has an AI coach. It reads the selected report and uses a Retrieval-Augmented Generation (RAG) pipeline with BGE embeddings, a ChromaDB store of cricket coaching manuals, a BGE reranker and a Qwen3:4B model that runs locally. A practice planner and a skills library with 36 skills complete the app. The scores are rule-based and we have not checked them against real coaches, so we do not claim any accuracy number in this work.

Keywords: Cricket Coaching, Computer Vision, Pose Estimation, MediaPipe, OpenCV, Retrieval-Augmented Generation, ChromaDB, Large Language Model, FastAPI, React.

FIGURE NO	TITLE	PAGE NO.
Figure 1	Overall architecture of CRICKETAI	5
Figure 2	Data flow from video upload to report and coaching	6
Figure 3	CRICKETAI landing page	10
Figure 4	Analysis page with discipline, bowler details and filming checklist	11
Figure 5	Video uploaded and analysis running on the server	11
Figure 6	Technique report with score, tracking confidence and biomechanics scores	12
Figure 7	Observations: strengths and areas to improve	12
Figure 8	Coaching note with corrections and drills	13
Figure 9	Practice planner showing the weakest area and corrective drills	14
Figure 10	Skills library with batting, bowling and fielding skills	14
Figure 11	AI coach page with the selected report and suggested questions	15
Figure 12	AI coach reviewing the report and coaching references	15
Figure 13	Pose Landmark Detection	18
Figure 14	Joint Angle Calculation	19
Figure 15	Spin Bowling Arm Path Evaluator	19
Figure 16	Technique Score and Tracking Confidence	20
Figure 17	Video Upload API Route	21
Figure 18	Report-Aware AI Coach Service	21
Figure 19	Cross-Encoder Reranker Loading	22
Figure 20	Local Qwen3:4B Model through Ollama	23
Figure 21	Planned theory and practical AI coach	26

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
BGE	BAAI General Embedding
CV	Computer Vision
FPS	Frames Per Second
HOD	Head of the Department
JWT	JSON Web Token
LLM	Large Language Model
ML	Machine Learning
ODM	Object Document Mapper
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
UI	User Interface
VISTAS	Vels Institute of Science, Technology and Advanced Studies

CHAPTER 1. INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

Cricket is played by lakhs of young players in schools, colleges and local grounds. To get better, a player needs someone to watch the batting stroke or the bowling action and tell what is wrong. Most players do not get this help every day. They practise alone and keep the same mistakes for a long time.

CRICKETAI is an AI cricket coach that runs as a web application. The player uploads one short video of a batting stroke or a bowling delivery. The system follows the body in every frame, scores each joint area and gives a full technique report. The same report is then used by an AI coach for questions, by a practice page for drills and by a progress page that shows the improvement over time.

Our project uses MediaPipe Pose [3], [4] to find body points in the video and a Retrieval-Augmented Generation (RAG) method [8] for the coach. The base paper for our work is the study by Siddiqui et al. [1], which showed that MediaPipe body points can describe cricket batting strokes well.

1.2 PROBLEM STATEMENT

A player can easily record practice on a phone, but the video alone does not say what to change. Coaching academies are costly and one coach has to handle many players. General chatbots can answer cricket questions, but they do not know the player's own technique and they may give confident but wrong advice [12].

This project tries to build one simple system that can do four things: (i) study a batting or bowling video with computer vision, (ii) give an easy report with scores, strengths, areas to improve and drills, (iii) answer the player's questions using the report and trusted coaching manuals, and (iv) guide the next practice with drills and skill lessons.

1.3 OBJECTIVES

- To build a web app where a player can sign in and upload a batting or bowling video.
- To find the body points in every frame using OpenCV and MediaPipe Pose.
- To score each joint area from 0 to 100 and group the scores by body part.
- To show a report with a technique score, tracking confidence, strengths, areas to improve, a coaching note and drills.

- To make an AI coach that reads the selected report and answers from cricket coaching manuals.
- To keep the AI model on our own computer, so that the player's video and questions are not sent to an outside AI service.
- To give a practice planner and a skills library for batting, bowling and fielding.
- To show how the system works with screenshots of the app.

1.4 SCOPE OF THE PROJECT

The present version covers batting and bowling. For bowling, the player can choose right arm or left arm, and pace or spin. One clip should have one stroke or one delivery, recorded with a single fixed camera. The app accepts MP4 or MOV files up to 200 MB. The skills library has 11 batting skills, 20 bowling skills and 5 fielding skills.

The scores in the report come from rule-based checks on the body points. We have not compared them with ratings by qualified coaches, so in this report we treat the score as a guide and not as a final judgement. Fielding videos are not analysed yet; fielding is only covered in the skills library.

1.5 APPLICATIONS AND EXPECTED OUTCOMES

- School, college and club players who want feedback on their own practice videos.
- Coaches who want a quick record of a player's posture between sessions.
- Academies that want a simple way to share drills and lessons with many players.
- Players who want to ask cricket technique questions at any time.
- Students who want to learn how computer vision and AI can be used in sports.

We expect the system to make technique feedback easy to get, to turn a phone video into clear points to work on, and to show how pose estimation, a report and an AI coach can work together. For real use, the scores need testing with coaches and with many more videos.

CHAPTER 2. LITERATURE REVIEW

2.1 BACKGROUND

Computer vision is now used in many sports to follow players, the ball and body movement [6]. Human pose estimation finds points such as the shoulders, elbows, hips, knees and ankles in an image. Deep learning methods such as OpenPose [7] made this possible in real time. BlazePose [4], which is used inside MediaPipe Pose [3], finds 33 body points for one person and runs fast even on a phone.

Our project uses the paper “Enhancing Cricket Performance Analysis with Human Pose Estimation and Machine Learning” by Siddiqui et al. [1] as the base paper. The authors used MediaPipe to take body features from videos of eight batting strokes, such as the pull, cut, cover drive and sweep, and trained several machine learning models. They reported a best accuracy of 99.77% with a Random Forest model. Their work shows that MediaPipe body points carry useful information about cricket batting.

2.2 EXISTING APPROACHES

Most cricket video studies try to find which stroke was played. Sen et al. [2] used a CNN with a GRU to classify batting shots from video clips. Siddiqui et al. [1] used body points and Random Forest for the same task. These systems answer “which shot was played” but not “how well was it played”, which is what a learner wants to know.

A review of camera-based pose estimation in sport [5] says that many studies do not check their results against a proper reference system. This is important because a single camera only sees a flat picture of a 3D movement.

For question answering, Lewis et al. [8] introduced Retrieval-Augmented Generation. In RAG, the system first finds useful passages from a document store and then gives them to a language model to write the answer. BGE embedding models [9] turn text into vectors for search, and a reranker [10] puts the most useful passages first. Small open models such as Qwen3 [11] can run on a normal computer. RAG reduces wrong answers but does not remove them fully [12].

2.3 RESEARCH GAP

Our review points to a gap between video analysis, explanation and practice. A system that only classifies the stroke does not tell the player what to fix. A

general chatbot can explain cricket but does not know the player's video. A list of drills on a website is not linked to the player's weak areas.

CRICKETAI tries to fill this gap in a small way. It links the pose analysis with a report, an AI coach that reads that report, and a practice page that picks drills for the weakest areas. The screenshots show these functions working together. A bigger study would need proper tests of the scores, the coach answers and the user experience.

2.4 PROPOSED CONTRIBUTION

- Pose-based analysis of batting and bowling videos with OpenCV and MediaPipe Pose.
- A joint-by-joint report with a technique score, tracking confidence, strengths, areas to improve and drills.
- An AI coach that reads the selected report and answers from cricket coaching manuals using RAG.
- A local language model, so the player's data stays on our own server.
- A practice planner and a skills library with 36 skills.

The main contribution is joining all these parts into one coaching workflow. The proof we have for this report is our project paper, our review presentation and the screenshots of the working app. We have not made any claim about numbers that we did not measure.

CHAPTER 3. SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

CRICKETAI has four connected layers: (1) the web app that the player uses, (2) the FastAPI server, (3) the analysis and AI coach engines, and (4) the data stores. The layers keep the video analysis separate from the coaching and practice parts, but the screens still show everything together. Figure 1 shows the overall architecture.

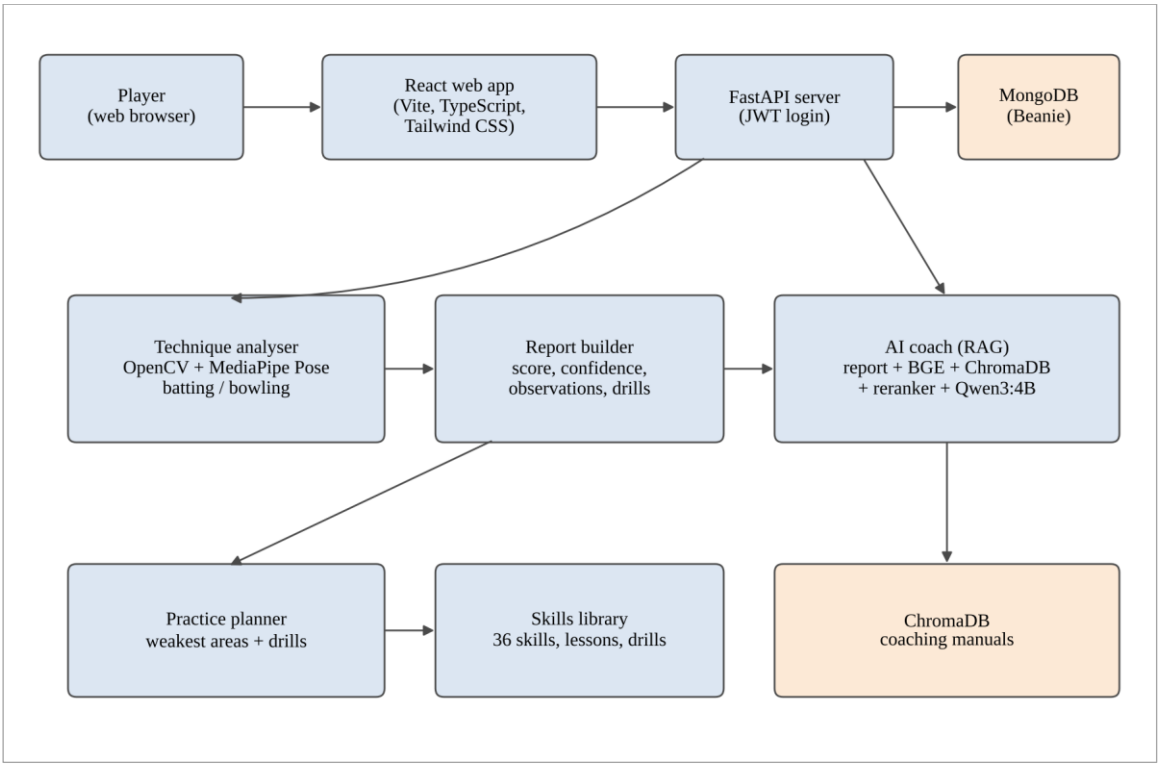


Figure 1 - Overall architecture of CRICKETAI

Table 1 - Layers of the system

Layer	Main part	Work done
Web app	React, Vite, TypeScript, Tailwind CSS	Shows the landing page, dashboard, upload, report, AI coach, practice, skills, progress and profile pages.
Server	FastAPI with JWT login	Checks the login, receives the video and sends data to the right engine.
Analysis engine	OpenCV and MediaPipe Pose	Reads the frames, finds 33 body points and scores each joint area.

Layer	Main part	Work done
Report builder	Scoring rules	Makes the technique score, tracking confidence, strengths, areas to improve, coaching note and drills.
AI coach	BGE, ChromaDB, BGE reranker, Qwen3:4B	Reads the selected report, finds useful manual passages and writes the answer.
Data	MongoDB (Beanie), video files, ChromaDB	Stores users, reports, videos and the coaching manual chunks.

3.1.1 Overall Data Flow

1. The player signs in and opens “New analysis”.
2. The player picks batting or bowling, the bowling arm and style, and uploads one clip.
3. The server reads the video frame by frame with OpenCV.
4. MediaPipe Pose finds 33 body points in each frame.
5. Each joint area is scored from 0 to 100 and the report is made.
6. The report is saved in MongoDB and shown to the player.
7. The AI coach and the practice page use the same report to answer questions and to pick drills.

Figure 2 shows this flow from the upload to the report, the coach and the practice page.

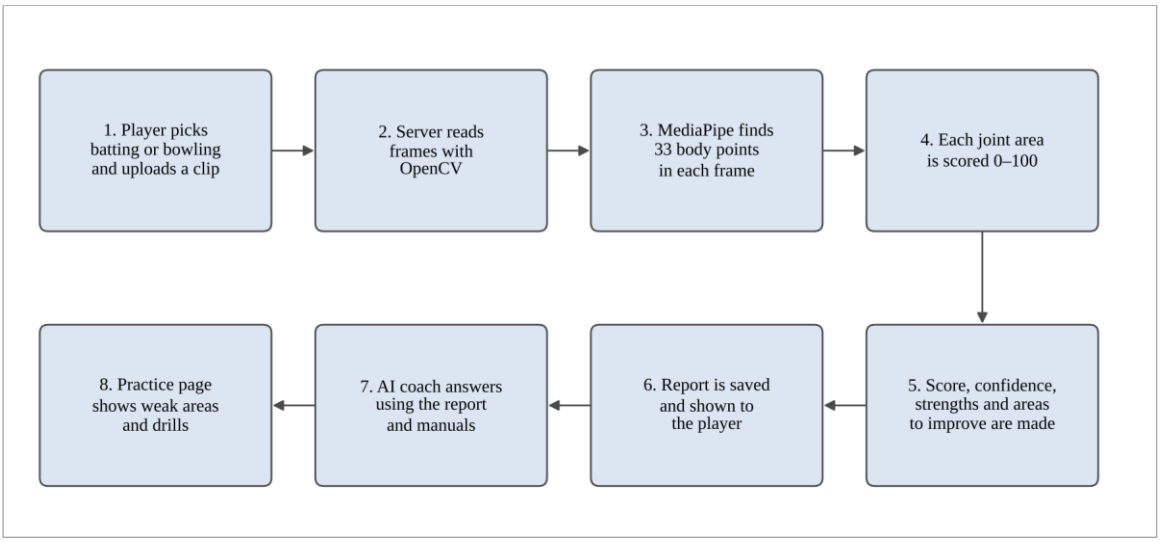


Figure 2 - Data flow from video upload to report and coaching

3.2 HARDWARE AND SOFTWARE REQUIREMENTS

The hardware and software used to build and run the project are given in Table 2.

Table 2 - Hardware and software requirements

Item	Requirement
Computer	Laptop or desktop with a multi-core processor; 16 GB RAM is better for running the local AI model
Camera	Smartphone camera that can record the full body at about 30 frames per second
Operating system	Windows (used for development)
Frontend	Node.js, React, Vite, TypeScript, Tailwind CSS
Backend	Python, FastAPI, Uvicorn, Pydantic
Database	MongoDB with the Beanie ODM
Computer vision	OpenCV, MediaPipe Pose
AI coach	ChromaDB, BAAI bge-base-en-v1.5, BGE reranker, Ollama with Qwen3:4B
Browser	Any modern browser such as Edge or Chrome

3.3 SOFTWARE ARCHITECTURE

The project has a React frontend and a Python backend. The frontend is built with Vite and TypeScript and styled with Tailwind CSS. The backend is a FastAPI server run by Uvicorn. Pydantic checks the data that comes into the server, and JWT tokens protect the pages that need a login. MongoDB stores the users and reports through the Beanie ODM. Table 3 lists the software parts.

Table 3 - Software parts of the system

Software part	Purpose in the system
Landing page	Introduces the app and leads to sign in, AI coach, practice planner, technique analysis and progress.
Dashboard and progress	Shows the player’s overview and weekly progress from all analysed sessions.

Software part	Purpose in the system
Analysis page	Collects the discipline, bowling arm, bowling style and video, and shows the progress of the analysis.
Report page	Shows the video with slow motion and frame step, the score, confidence, body-part scores, observations, coaching note and drills.
AI coach page	Lets the player pick a report and ask questions; shows suggested questions from the report.
Practice page	Shows the lowest-scoring areas of the latest session and the drills for them.
Skills library	Lists batting, bowling and fielding skills with level filter, search, lessons and drills.

3.4 DATA AND ANALYSIS FLOW

The system must keep two ideas apart: what was measured from the video, and what that means for coaching. The body points and the angles between them are measurements of the picture. Whether an angle is good or bad depends on the stroke, the phase of the stroke, the player and the camera angle. So the report shows the measured scores and also a tracking confidence value, which tells how reliably the body was followed in the clip.

Our report screenshot shows the scores grouped by body part, such as Head & Balance, Arms, Hands & Bat, and Rotation & Sequencing. The colours show 75 and above as strong, 50 to 74 as developing and below 50 as priority. The lowest scores are shown first, so the player sees the weakest area at once.

3.5 AI COACH DESIGN

The AI coach uses Retrieval-Augmented Generation [8]. Cricket coaching manuals are cut into small text chunks. Each chunk is turned into a vector with the BAAI bge-base-en-v1.5 model [9] and stored in ChromaDB. Our store has about 1,002 chunks.

When the player asks a question, the question is also turned into a vector and the closest chunks are found in ChromaDB. A BGE cross-encoder reranker (BAAI/bge-reranker-base) [10] reads the question with each chunk and puts the most useful chunks first. The top chunks, the player's selected report and the question are given to the Qwen3:4B model [11], which runs on our computer

through Ollama. The model is told to answer from this information. Because the model runs locally, an answer can take a minute or two on a laptop.

CHAPTER 4. IMPLEMENTATION

4.1 LANDING PAGE AND SESSION UPLOAD

The landing page introduces CRICKETAI as “Your AI Cricket Coach”. The top menu has Home, AI Coach, Practice Planner, Technique Analysis and Progress, with Sign in and Get Started buttons. The example analysis card on the right is marked as illustrative values, so the player knows these are not real results.

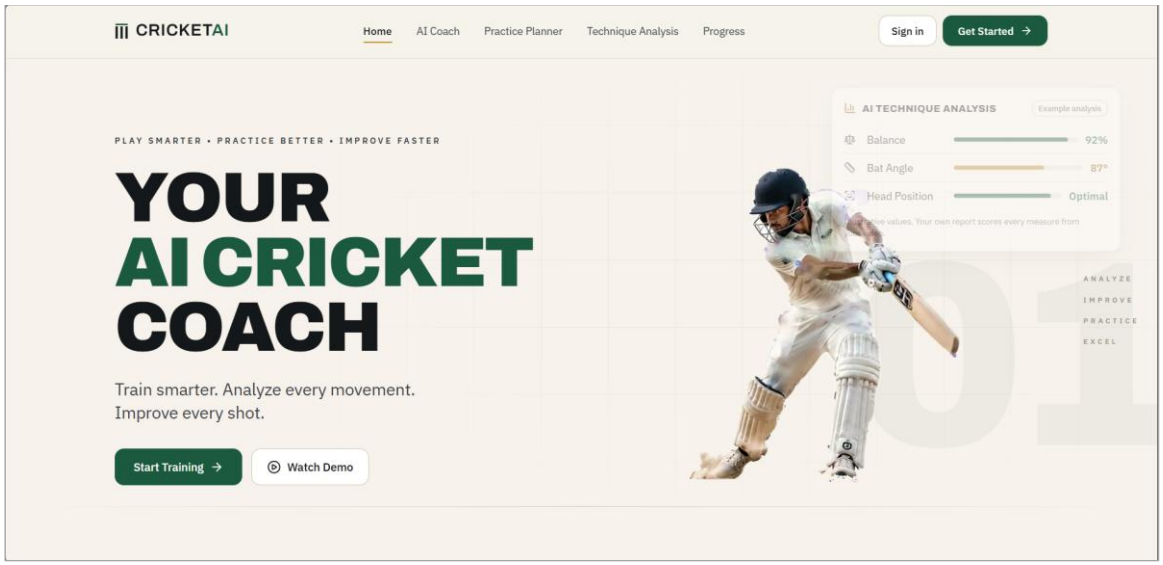


Figure 3 - CRICKETAI landing page

After signing in, the player uses the side menu to open Dashboard, Analysis, AI Coach, Practice, Skills, Progress and Profile. The “New analysis” button opens the upload page shown in Figure 4. The page has three steps: Discipline (batting or bowling), Bowler (right arm or left arm, pace or spin) and Video (MP4 or MOV up to 200 MB). A filming checklist on the right tells the player to keep the whole body in the frame, fix the camera, use good light, record one stroke or delivery per clip and wear clothing that contrasts with the background.

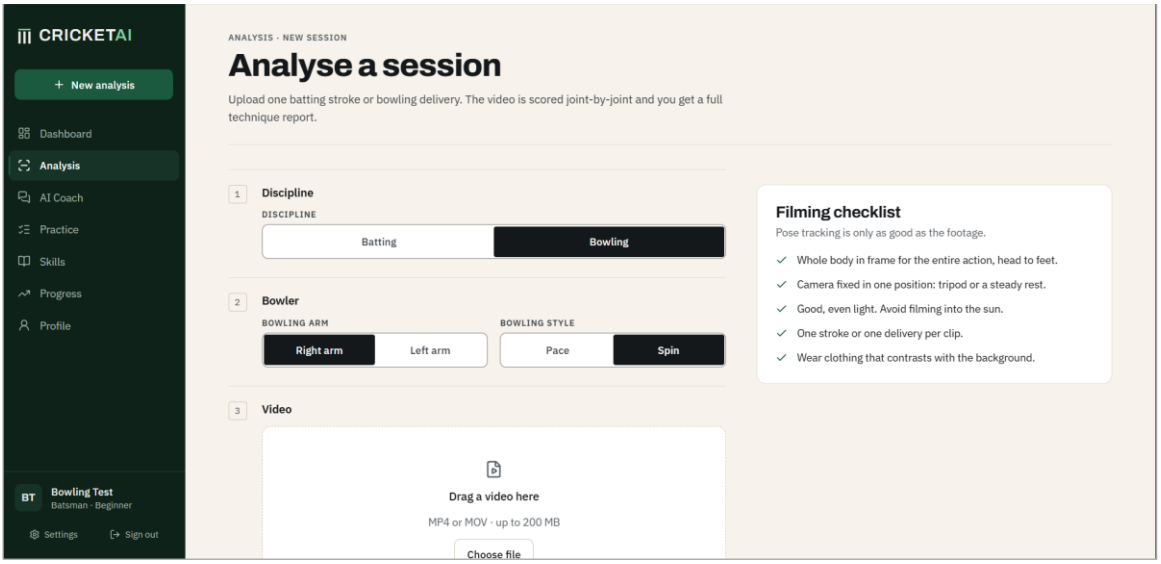


Figure 4 - Analysis page with discipline, bowler details and filming checklist

Figure 5 shows the page after a 13-second clip is chosen. The message "Analysing technique on the server" and a progress bar are shown while the server tracks the pose frame by frame and scores each joint. The player is asked to keep the page open.

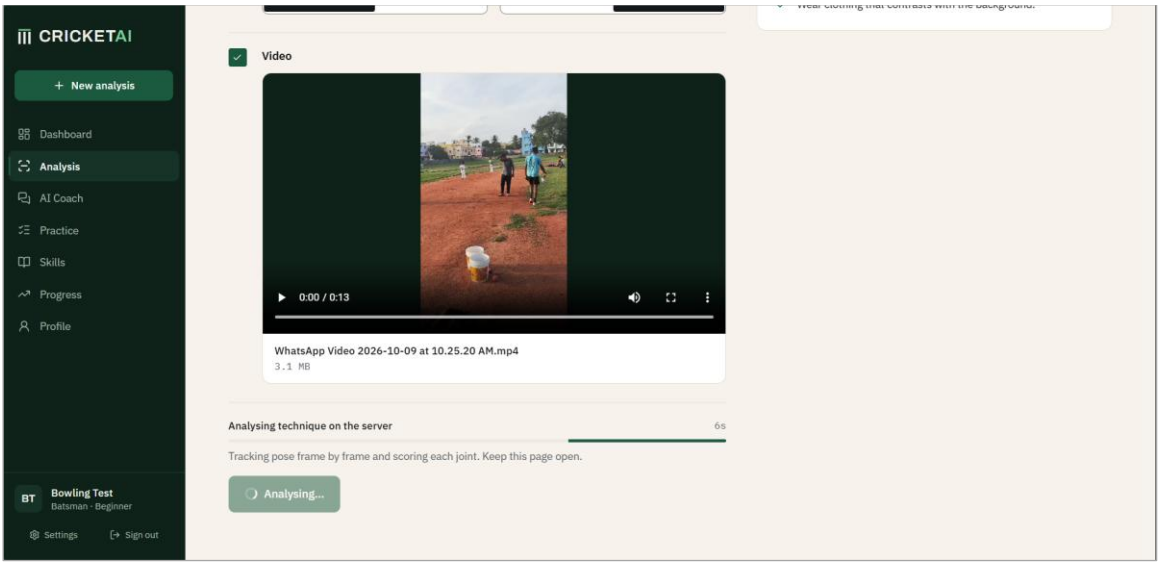


Figure 5 - Video uploaded and analysis running on the server

4.2 TECHNIQUE REPORT

When the analysis is complete, the app shows "Analysis complete. Here's your report." The left side has the video player with frame step and 0.25×, 0.5× and 1× speed buttons. Below it, the technique score card shows the score out of 100, a label such as STRONG, the tracking confidence, the recorded time and the

discipline. The right side has five parts: Summary, Biomechanics, Observations, Coaching and Drills.

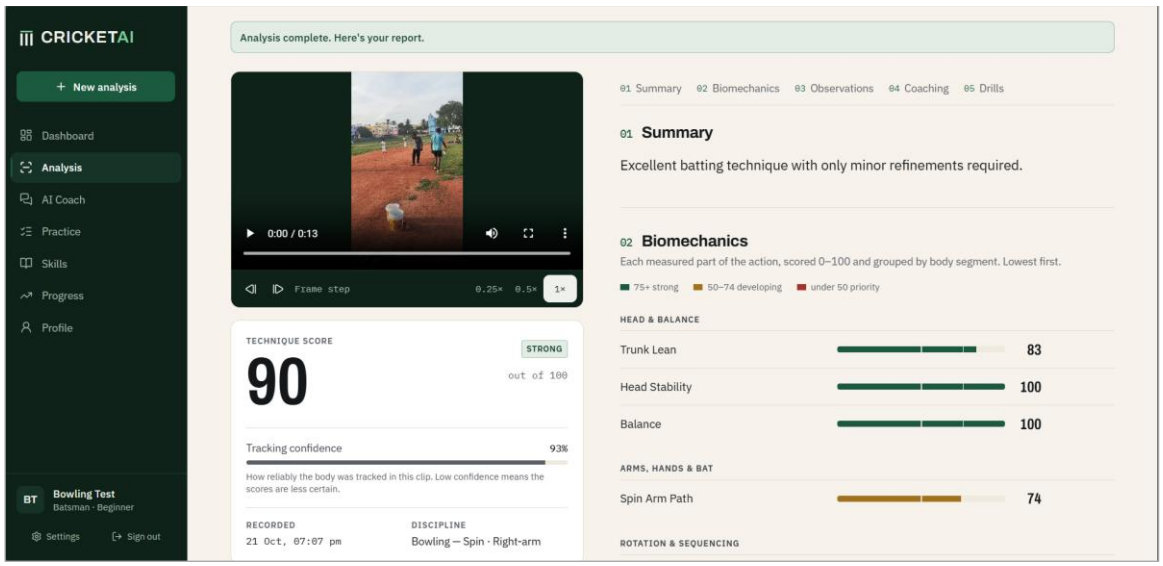


Figure 6 - Technique report with score, tracking confidence and biomechanics scores

Figure 6 shows our sample spin bowling clip (right arm). The technique score is 90 out of 100 and the tracking confidence is 93%. In the Biomechanics part, Head Stability and Balance are 100, Trunk Lean is 83 and Spin Arm Path is 74. Spin Arm Path is shown in the developing colour because it is between 50 and 74.

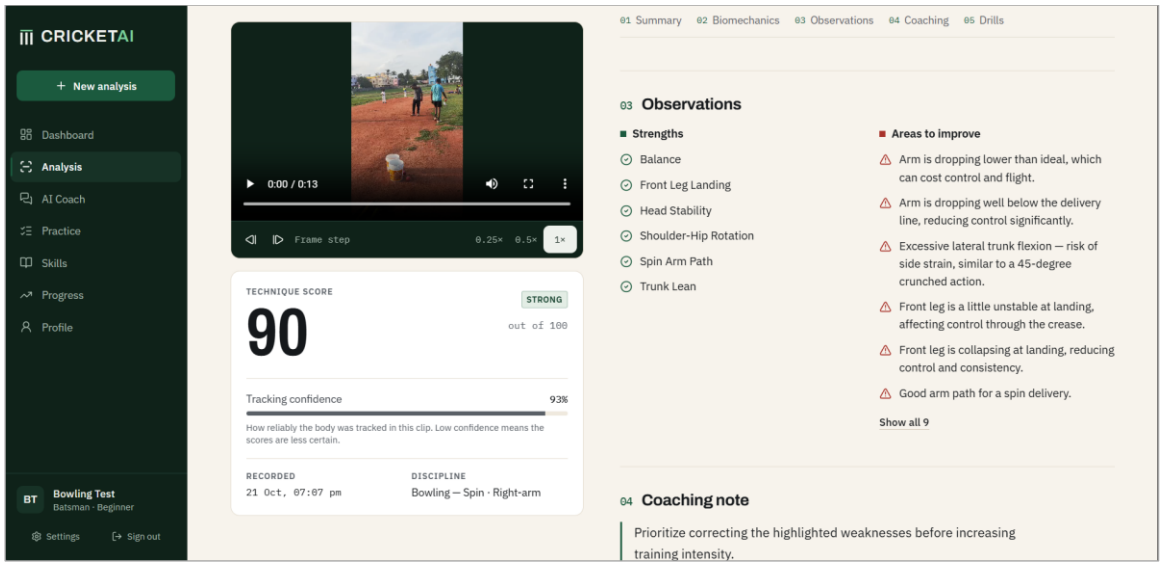


Figure 7 - Observations: strengths and areas to improve

Figure 7 shows the Observations part. The strengths list Balance, Front Leg Landing, Head Stability, Shoulder-Hip Rotation, Spin Arm Path and Trunk Lean. The areas to improve explain the problems in simple words, for example

that the arm is dropping lower than ideal, which can cost control and flight. The “Show all 9” link opens the full list.

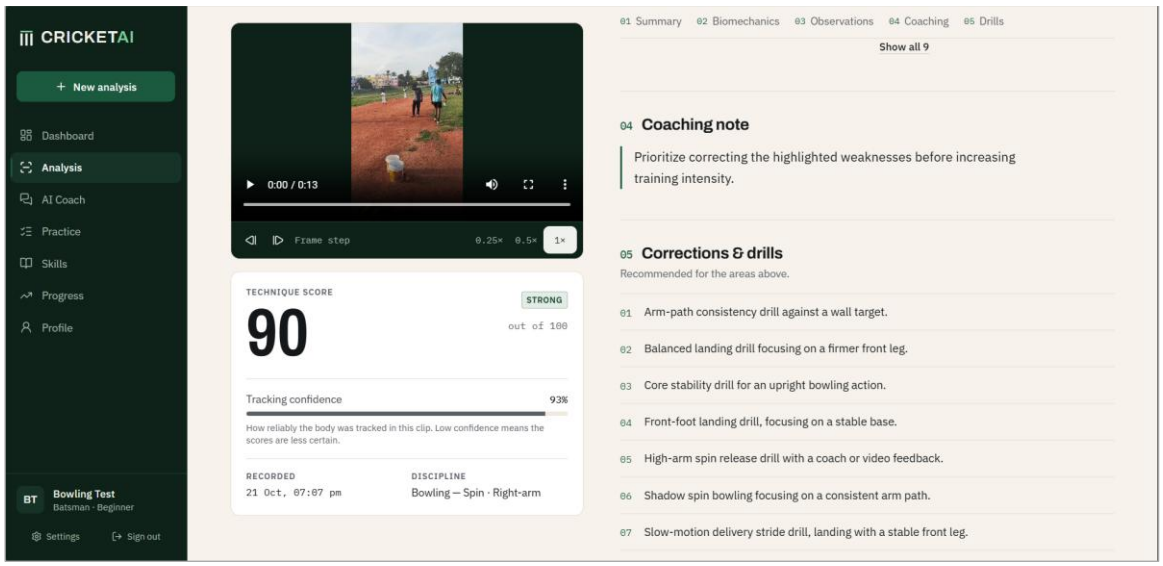


Figure 8 - Coaching note with corrections and drills

Figure 8 shows the coaching note and the corrections and drills. The note asks the player to correct the highlighted weaknesses before increasing the training intensity. The drills include an arm-path consistency drill against a wall target, a balanced landing drill, a core stability drill and a high-arm spin release drill with coach or video feedback.

4.3 PRACTICE PLANNER AND SKILLS LIBRARY

The Practice page, called Training focus, tells the player what to work on, why and how, based on the latest analysed session. Part 01 shows the lowest-scoring areas of the session. In our example it is Spin Arm Path with a score of 74, from the bowling session of 21 Oct 2026. Part 02 lists the nine corrective drills prescribed in the report.

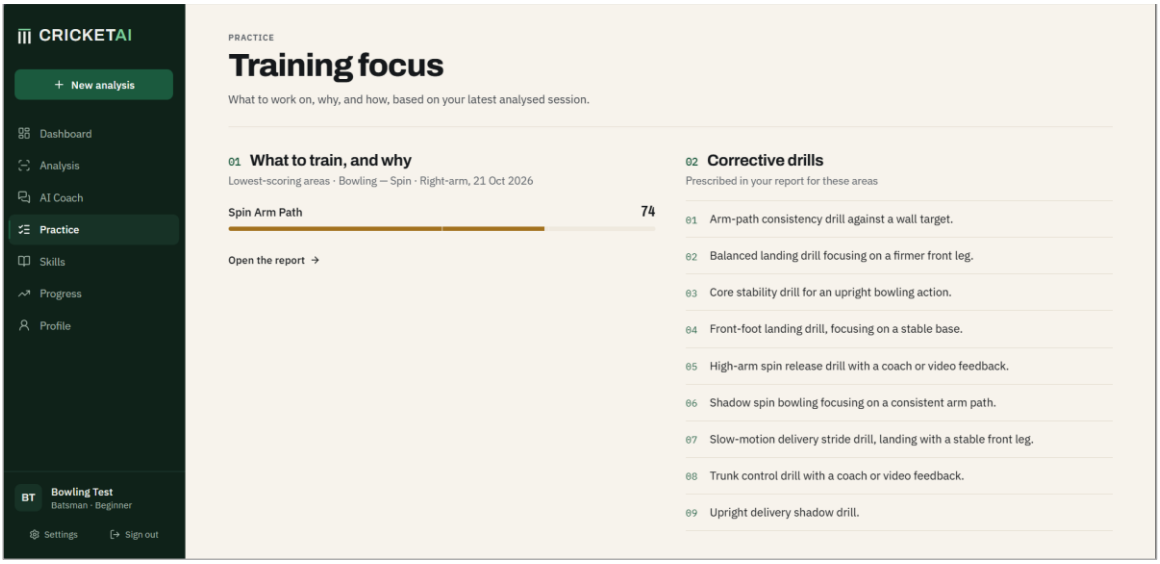


Figure 9 - Practice planner showing the weakest area and corrective drills

The Skills page is a library of cricket skills. It has three tabs: Batting (11), Bowling (20) and Fielding (5). The player can search, for example “drive”, “yorker” or “catching”, and filter by Beginner, Intermediate or Advanced. Each card shows the skill name, a short description, the level and the number of lessons and drills. For example, Cover Drive has 4 lessons and 2 drills.

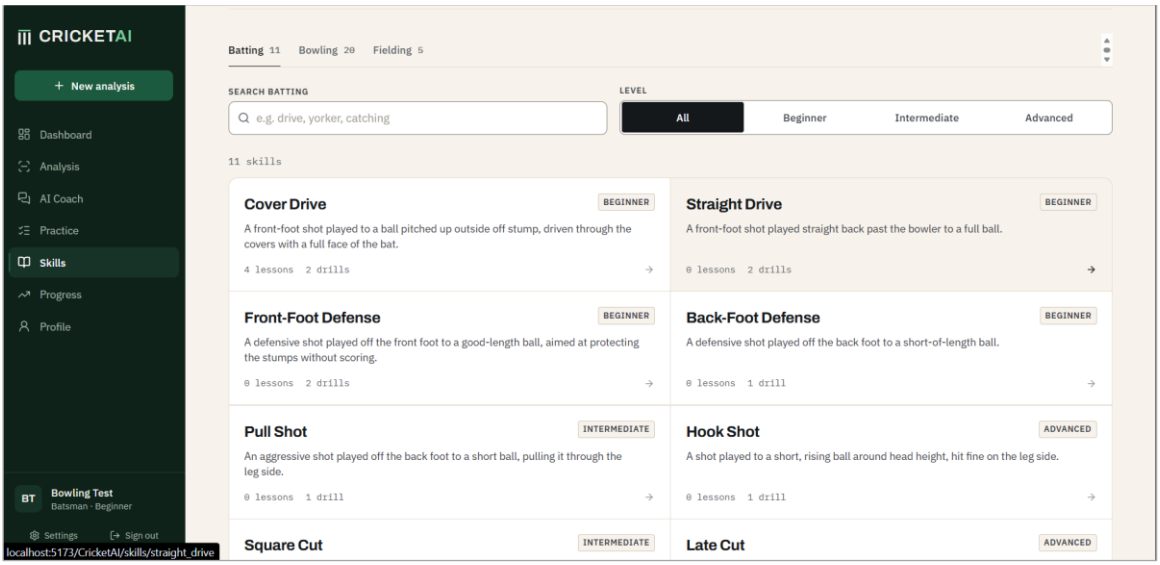


Figure 10 - Skills library with batting, bowling and fielding skills

4.4 AI COACH WORKFLOW

The AI Coach page opens a coaching session. The heading says that the coach has read the selected report and draws on cricket coaching manuals to answer. The player selects a report in the “Coaching on” box. The card below shows the technique score, the technique issues and the coach recommendation from that report. The coach also gives suggested questions drawn from the report, such as

“What should I fix first in this session?” and “How do I improve my spin arm path?”.

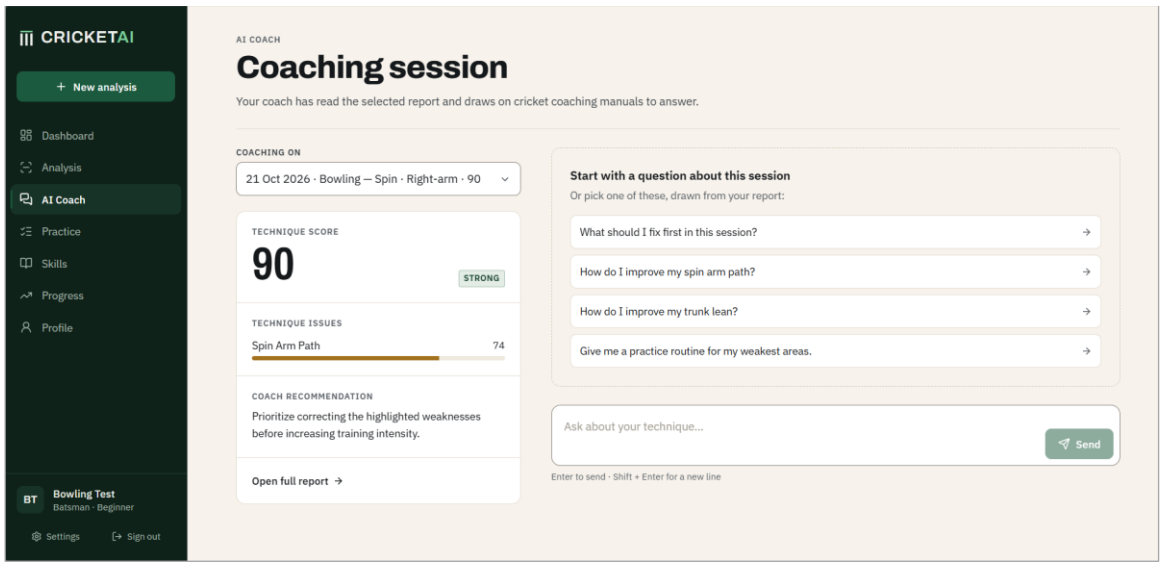


Figure 11 - AI coach page with the selected report and suggested questions

Figure 12 shows the screen after the player asks “How do I improve my spin arm path?”. The coach shows “Reviewing your report and the coaching references…” with a timer. A note says that the coach model runs locally and that answers can take a minute or two.

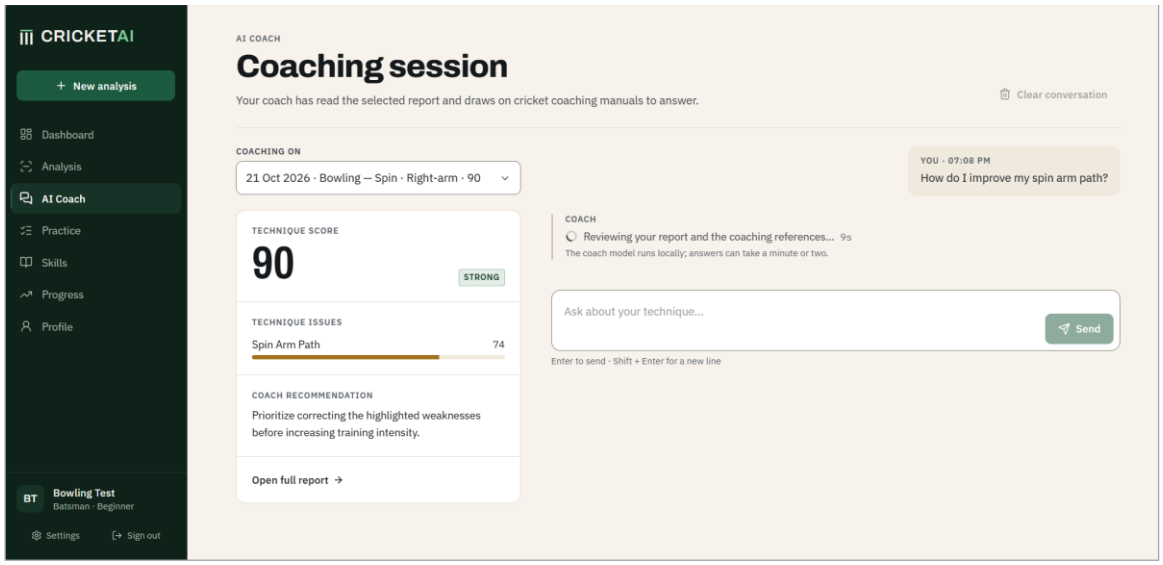


Figure 12 - AI coach reviewing the report and coaching references

4.4.1 Step-by-Step Coaching Session

1. Sign in and click “New analysis”.
2. Choose batting or bowling. For bowling, choose the bowling arm and the style.

3. Upload one clip and wait while the server analyses the technique.
4. Read the report: the score, confidence, body-part scores, observations, coaching note and drills.
5. Open the AI Coach and select the report in the “Coaching on” box.
6. Click a suggested question or type a question and press Enter.
7. Read the answer, then open the Practice page to follow the drills.
8. Upload a new clip later and check the Progress page for improvement.

4.5 CORE LOGIC SUMMARY

The logic below is a short summary of how the system works. It is not copied from the source code. We wrote it in plain steps to explain the working.

Technique analysis

1. Receive the video with the discipline, bowling arm and style.
2. Read each frame with OpenCV and change it from BGR to RGB colour order.
3. Find the 33 body points with MediaPipe Pose and keep their visibility values.
4. Work out the angles and positions needed for each joint area.
5. Score each joint area and work out the tracking confidence as the share of frames in which the body was found.

Report building

1. Group the scores by body part and sort them with the lowest first.
2. Make the overall technique score as a weighted average, giving more weight to areas such as head stability and balance.
3. List the strengths and the areas to improve.
4. Add the coaching note and pick the drills for the weak areas.
5. Save the report for the player and show it on the screen.

AI coach answer

1. Take the selected report and the player’s question.
2. Turn the question into a vector with the BGE model.
3. Find the closest manual chunks in ChromaDB.
4. Rerank the chunks and keep the best ones.
5. Give the report, the chunks and the question to Qwen3:4B and show the answer.

4.6 TESTING AND OBSERVATIONS

Table 4 lists what we observed in the demo and the figure that shows it.

Table 4 - Test cases and observations

Test case	Figure	Observation
Landing page	Figure 3	Home page opens with menu, Sign in, Get Started and Start Training buttons.
Upload inputs	Figure 4	Discipline, bowling arm, style and video steps are shown with the filming checklist.
Server analysis	Figure 5	Progress message is shown while the pose is tracked and the joints are scored.
Report generation	Figure 6	Score 90 out of 100 (STRONG) and 93% tracking confidence for the sample spin clip.
Body-part scores	Figure 6	Head Stability 100, Balance 100, Trunk Lean 83, Spin Arm Path 74.
Observations and drills	Figures 7, 8	Strengths, areas to improve, coaching note and drills are shown.
Practice planner	Figure 9	Lowest area (Spin Arm Path, 74) and nine drills are shown.
Skills library	Figure 10	11 batting, 20 bowling and 5 fielding skills with level filter and search.
AI coach with report	Figure 11	Selected report and suggested questions from the report are shown.
AI coach answer	Figure 12	Coach reviews the report and references; local model note is shown.
Server API checks	-	Login, dashboard overview, weekly progress, video upload, video retrieval and coach query returned HTTP 200.
Long batting clip	-	All 1,654 frames of a clip recorded at about 30 frames per second were processed.

These observations show that the upload, analysis, report, practice, skills and AI coach screens work in the demo. We did not record an accuracy test of the

scores, the time taken for each analysis, a check of the coach answers by a coach or a user study. So we do not give numbers for these.

4.7 SOURCE CODE IMPLEMENTATION

This section shows the core code files of our project as plain text. Each code part has its file name and a short explanation. Long files are shortened and the cut parts are marked with "...". The code is linked to the results in Section 4.8. The environment file is not included because it has private settings.

Figure 13 - Pose Landmark Detection

File Name: backend/app/vision/pose_detector.py

```
class PoseDetector:
    def __init__(self):
        self.mp_pose = mp.solutions.pose
        self.pose = self.mp_pose.Pose(
            static_image_mode=True,
            model_complexity=1,
            enable_segmentation=False,
            min_detection_confidence=0.5,
        )
    def process_frames(
        self,
        frames_dir: str,
        output_json: str,
    ) -> dict:
        frames_dir = Path(frames_dir)
        output_json = Path(output_json)
        output_json.parent.mkdir(parents=True, exist_ok=True)
        results_json = []
        processed = 0
        detected = 0
        frame_files = sorted(frames_dir.glob("*.jpg"))
        for frame_file in frame_files:
            image = cv2.imread(str(frame_file))
            if image is None:
                continue
            rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
            result = self.pose.process(rgb)
            frame_data = {
                "frame": frame_file.name,
                "landmarks": [],
            }
            if result.pose_landmarks:
                detected += 1
                for idx, lm in enumerate(result.pose_landmarks.landmark):
                    frame_data["landmarks"].append({
                        "id": idx,
                        "x": lm.x,
                        "y": lm.y,
                        "z": lm.z,
                        "visibility": lm.visibility,
                    })
            ...
```

Explanation: This class loads MediaPipe Pose and reads every saved frame. Each frame is changed from BGR to RGB, and the 33 body points with their x, y, z and visibility values are stored for the frame.

Figure 14 - Joint Angle Calculation

File Name: backend/app/biomechanics/common/angles.py

```
class AngleCalculator:
    @staticmethod
    def calculate_angle(a, b, c):
        """
        Calculate the angle ABC (in degrees).
        Parameters:
            a, b, c -> {"x": float, "y": float} landmark points
        Returns:
            Angle in degrees.
        """
        ba = (a["x"] - b["x"], a["y"] - b["y"])
        bc = (c["x"] - b["x"], c["y"] - b["y"])
        dot = ba[0] * bc[0] + ba[1] * bc[1]
        mag_ba = math.sqrt(ba[0] ** 2 + ba[1] ** 2)
        mag_bc = math.sqrt(bc[0] ** 2 + bc[1] ** 2)
        if mag_ba == 0 or mag_bc == 0:
            return None
        cosine = dot / (mag_ba * mag_bc)
        cosine = max(-1.0, min(1.0, cosine))
        angle = math.degrees(math.acos(cosine))
        return round(angle, 2)
```

Explanation: This function finds the angle at the middle point B from three body points A, B and C. It uses the dot product of the two arms of the angle and limits the cosine value between -1 and 1 before taking the inverse cosine.

Figure 15 - Spin Bowling Arm Path Evaluator

File Name: backend/app/biomechanics/evaluators/bowling_arm_spin.py

```
class SpinArmPathEvaluator:
    IDEAL_MIN = 130.0
    IDEAL_MAX = 170.0
    GOOD_MIN = 110.0
    GOOD_MAX = 129.9
    FAIR_MIN = 90.0
    FAIR_MAX = 109.9
    def evaluate(
        self,
        arm_angle: float,
    ) -> MetricEvaluation:
        if arm_angle >= self.IDEAL_MIN:
            score = 10
            status = MetricStatus.EXCELLENT
            severity = Severity.NONE
            feedback = "Consistent, well-controlled arm path through the
delivery."
            drill = None
        elif arm_angle >= self.GOOD_MIN:
            score = 8
            status = MetricStatus.GOOD
            severity = Severity.LOW
```

```
            feedback = "Good arm path for a spin delivery."
            drill = "Shadow spin bowling focusing on a consistent arm
path."
        elif arm_angle >= self.FAIR_MIN:
            score = 6
            status = MetricStatus.FAIR
            severity = Severity.MEDIUM
            feedback = "Arm is dropping lower than ideal, which can cost
control and flight."
            drill = "Arm-path consistency drill against a wall target."
        ...
```

Explanation: This evaluator checks the bowling arm angle for spin bowling. It gives a score, a status, a feedback sentence and a drill for each band of the angle. These texts are the ones seen in the Observations and Drills parts of the report.

Figure 16 - Technique Score and Tracking Confidence

File Name: backend/app/biomechanics/evaluators/scoring.py

```
    DEFAULT_WEIGHT = 5
    METRIC_WEIGHTS = {
        "Head Stability": 10,
        "Balance": 10,
        "Front Elbow": 9,
        "Front Knee": 9,
        "Back Elbow": 8,
        "Shoulder Alignment": 8,
        "Hip Rotation": 8,
        "Back Knee": 7,
        "Stance": 7,
    }
    def calculate_overall_score(
        self,
        metrics: list[MetricEvaluation],
    ) -> int:
        if not metrics:
            return 0
        weighted_score = 0.0
        total_weight = 0.0
        for metric in metrics:
            weight = self.METRIC_WEIGHTS.get(
                metric.metric,
                self.DEFAULT_WEIGHT,
            )
            weighted_score += metric.score * weight
            total_weight += weight
        if total_weight == 0:
            return 0
        return round((weighted_score / total_weight) * 10)
    def calculate_confidence(
        self,
        total_frames: int,
        valid_frames: int,
    ) -> int:
        if total_frames == 0:
            return 0
        return round(
            (valid_frames / total_frames) * 100
```

```
        )
```

Explanation: The technique score is a weighted average of the metric scores, where important areas such as head stability and balance get more weight. The tracking confidence is the share of frames in which the body was found, shown as a percentage.

Figure 17 - Video Upload API Route

File Name: backend/app/api/analysis.py

```
@router.post(
    "/upload",
    response_model=AnalysisResponse,
    status_code=status.HTTP_201_CREATED,
)
async def upload_video(
    video: UploadFile = File(...),
    current_user=Depends(get_current_user),
):
    if not video.content_type.startswith("video/"):
        raise HTTPException(
            status_code=400,
            detail="Only video files are allowed.",
        )
    analysis = await analysis_service.analyze_video(
        user_id=str(current_user.id),
        video=video,
    )
    ...
```

Explanation: This FastAPI route accepts the uploaded video only for a logged-in user. It rejects files that are not videos and sends the video to the analysis service, which returns the saved report.

Figure 18 - Report-Aware AI Coach Service

File Name: backend/app/rag/chat_service.py

```
    async def ask(
        self,
        *,
        user_id: str,
        analysis_id: str,
        question: str,
    ) -> str:
        report = await self.analysis_repo.get_by_id(
            analysis_id
        )
        # Same error for a foreign analysis as for a missing one, so the
        # response doesn't reveal that another user's analysis exists.
        if report is None or report.user_id != user_id:
            raise ValueError(
                "Analysis not found."
            )
        history = await self.memory.get_history(
            user_id=user_id,
            analysis_id=analysis_id,
        )
        chunks = self.retriever.retrieve(
```

```
            question=question,
        )
        prompt = PromptBuilder.build(
            report=report,
            retrieved_chunks=chunks,
            conversation_history=history,
            user_question=question,
        )
        answer = await self.qwen.ask(
            prompt
        )
        await self.memory.add_message(
            user_id=user_id,
            analysis_id=analysis_id,
            role="user",
            content=question,
        )
        await self.memory.add_message(
            user_id=user_id,
            analysis_id=analysis_id,
            role="assistant",
            content=answer,
        )
        return answer
```

Explanation: This service first loads the selected report and checks that it belongs to the user. It then gets the past messages, retrieves the useful manual chunks, builds the prompt with the report and asks the Qwen model. The question and the answer are saved for the next turn.

Figure 19 - Cross-Encoder Reranker Loading

File Name: backend/app/rag/retrieval/reranker.py

```
def get_reranker() -> CrossEncoder:
    """
    Load the CrossEncoder model once (Singleton).
    Automatically uses GPU if CUDA is available,
    otherwise falls back to CPU.
    """
    global _MODEL
    if _MODEL is None:
        device = "cuda" if torch.cuda.is_available() else "cpu"
        print("Loading reranker...")
        print(f"Using device: {device}")
        _MODEL = CrossEncoder(
            "BAAI/bge-reranker-base",
            device=device,
        )
    return _MODEL
```

Explanation: This function loads the BAAI/bge-reranker-base cross-encoder only once and uses the GPU when one is available. The reranker reads the question with each retrieved chunk and puts the most useful chunks first.

Figure 20 - Local Qwen3:4B Model through Ollama

File Name: backend/app/rag/llm/ollama_provider.py

```
class OllamaProvider(BaseLLMProvider):
```

```
def __init__(
    self,
    model: str = "qwen3:4b",
    host: str = "http://localhost:11434",
):
    self.model = model
    self.client = AsyncClient(
        host=host,
    )
async def generate(
    self,
    prompt: str,
) -> str:
    response = await self.client.chat(
        model=self.model,
        messages=[
            {
                "role": "user",
                "content": prompt,
            }
        ],
        options={
            "temperature": 0.2,
            "top_p": 0.9,
            "num_predict": 2048,
        },
    )
    return response["message"]["content"].strip()
```

Explanation: This class sends the final prompt to the qwen3:4b model running in Ollama on the local computer. A low temperature of 0.2 keeps the answers focused.

4.8 RESULTS AND DISCUSSION

The app shows a clear path from one uploaded clip to a full technique report. The report gives a quick summary with the score and confidence, and also shows each joint area separately. The observations turn the numbers into simple sentences, and the drills give the next step. The practice page brings the weakest area and its drills to the front, so the player knows what to do in the next session.

The tracking confidence is an important part of the result. In our sample clip it was 93%, which means the body was followed well. A low value tells the player that the scores are less certain and that the clip should be recorded again with a better camera position. The filming checklist on the upload page helps to avoid such problems.

The AI coach is the part that links everything. Because it reads the selected report, its suggested questions and answers are about the player's own weak areas, not general cricket tips. The answers come from coaching manuals through RAG, but a 4-billion-parameter model can still make mistakes [12], so

the answers should be checked by a coach. A full test should report the score accuracy against coaches, the analysis time, the quality of the retrieved manual passages and the coach answers, and the opinion of real players.

Table 5 links each result to the code files that gave the output.

Table 5 - Results and the code files behind them

Result	Figure	Main code file(s)
Session upload and analysis	Figures 4, 5, 13, 17	analysis.py, pose_detector.py
Joint angles and spin arm path	Figures 6, 14, 15	angles.py, bowling_arm_spin.py
Technique score and tracking confidence	Figures 6, 16	scoring.py
Observations, coaching note and drills	Figures 7, 8, 9, 15	bowling_arm_spin.py, scoring.py
Report-aware AI coach answer	Figures 11, 12, 18, 19, 20	chat_service.py, reranker.py, ollama_provider.py

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

CRICKETAI is an AI cricket coaching web application that joins pose-based video analysis, a technique report, an AI coach, a practice planner and a skills library. The upload screenshots show how a batting or bowling clip is given to the system. The report screenshots show the score, the tracking confidence, the body-part scores, the observations and the drills. The AI coach screenshots show that the coach reads the selected report before answering.

The system helps with a real coaching problem. It turns a simple phone video into clear points to work on, and it lets the player ask questions at any time. The AI model runs on our own computer, so the player's video and questions are not sent to an outside AI service.

The scores are made by rules on the body points, and we have not checked them against qualified coaches. From our demo, we can say that the full workflow works. More testing is needed before we can say anything about the accuracy of the scores or the quality of the coach answers.

5.2 LIMITATIONS

- A single camera gives a flat picture, so the measured angles change with the camera angle, distance and light.
- The bat, pads and nets can hide some joints and reduce the tracking confidence.
- We do not have a checked accuracy value for the scores, so we do not claim any.
- The local AI model can take a minute or two to answer and can still give a wrong answer.
- Fielding videos are not analysed yet, and some skills in the library have drills but no lessons yet.

5.3 FUTURE ENHANCEMENTS

Today our AI coach works on theory knowledge. Its knowledge base is made from coaching manuals and books, which explain how a shot such as the cover drive should be played. The coach reads the player's report and gives feedback from this written knowledge. A real coach, however, also uses practical knowledge that comes from years of watching and playing the game. Our main future goal is to add this practical knowledge, so that CRICKETAI works as a theory and practical AI coach.

Theory and Practical AI Coach

We plan to record experienced state-level players who have played for about 10 years. For every shot and every type of delivery, we will collect about 200 to 400 videos. These videos will be used to train the system, so that it learns how expert players really move for each shot, and not only how the books describe it. Figure 21 shows how the present theory part and the planned practical part will join.

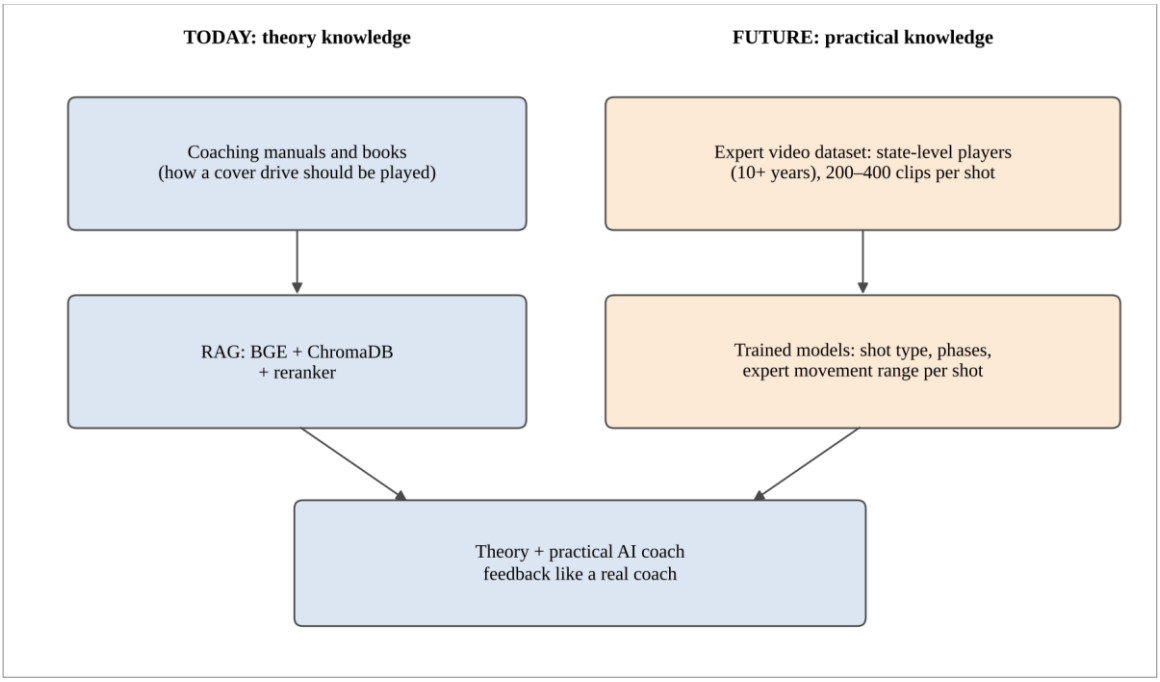


Figure 21 - Planned theory and practical AI coach

1. Record each shot and delivery of the expert players with a fixed camera, from the same angles that our users will use.
2. Ask qualified coaches to label each clip with the shot type, the phases of the shot and the quality of the technique.
3. Train models to find the shot type and the phases, such as the backlift, downswing, impact and follow-through.
4. Learn the normal movement range of the experts for every joint, at every phase of every shot.
5. Compare the learner's movement with the expert range, instead of using only fixed rules, to give the score.
6. Give the AI coach both kinds of knowledge: the manuals for the theory and the expert data for the practice.

With both kinds of knowledge, the coach can give feedback like a real coach, for example: "At impact, your front elbow is lower than in most expert cover drives; keep the elbow high and lead with the shoulder." Before using this in the

real world, we will check the new scores with qualified coaches and take written consent from every player who is recorded.

Other Enhancements

- Add fielding analysis for catching and throwing.
- Build a mobile app that gives live feedback through the phone camera.
- Use two or more cameras or 3D pose to measure the true joint angles.
- Add ball and bat tracking.
- Show the player's body next to an expert player's movement from our dataset for comparison.
- Add a voice coach in Tamil, Hindi and English.
- Make training plans that change with the weekly progress, and a portal for coaches and academies.

5.4 FINAL REMARK

CRICKETAI gives a practical base for an AI cricket coach. It joins video analysis, a clear report, an AI coach and guided practice in one place. This shows how computer vision and AI can make good coaching easier to reach. The next step is to add practical knowledge from experienced players, so that the AI coach can guide a player like a real coach, and to test the system properly with coaches and players before real use.

Appendix A - Conference paper Certificates



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

QS I-GAUGE PLATINUM | NAAC A++ GRADE | nirf 45 RANK

CERTIFICATE OF PARTICIPATION

This certificate is presented to

ABDUL WAHID H

Vels Institute of Science, Technology & Advanced Studies, Chennai

for actively participating in **AICTE** Sponeored **International Conference on Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

CRICKETAI: AN INTELLIGENT VIRTUAL CRICKET COACH.

held during 21.10.2026 & 22.10.2026 erganized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have ccntributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
Dr. B. Ramesh

Certificate 1 – Abdul Wahid H



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

QS I-GAUGE PLATINUM | NAAC A++ GRADE | nirf 45 RANK

CERTIFICATE OF PARTICIPATION

This certificate is presented to

SARVESHWARAN K

Vels Institute of Science, Technology & Advanced Studies, Chennai

for actively participating in **AICTE** Sponsored **International Conference on Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

CRICKETAI: AN INTELLIGENT VIRTUAL CRICKET COACH.

held during 21.10.2026 & 22.10.2026 erganized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have ccntributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
Dr. B. Ramesh

Certificate 2 – Sarveshwaran K



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

PLATINUM

CERTIFICATE OF PARTICIPATION

This certificate is presented to

VIGNESH P

Vels Institute of Science, Technology & Advanced Studies, Chennai

for actively participating in **AICTE** Sponsored **International Conference on Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

CRICKETAI: AN INTELLIGENT VIRTUAL CRICKET COACH.

held during 21.10.2026 & 22.10.2026 erganized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have ccntributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
Dr. B. Ramesh

Certificate 3 – Vignesh P



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

PLATINUM

CERTIFICATE OF PARTICIPATION

This certificate is presented to

Dr. N. Meenakshi

Vels Institute of Science, Technology & Advanced Studies, Chennai

for actively participating in **AICTE** Sponsored **International Conference on Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

CRICKETAI: AN INTELLIGENT VIRTUAL CRICKET COACH.

held during 21.10.2026 & 22.10.2026 erganized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have ccntributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
Dr. B. Ramesh

Certificate 4 – Dr. N Meenakshi

REFERENCES

[1] A. Badiola-Bengoa and A. Mendez-Zorrilla, "A systematic review of the application of camera-based human pose estimation in the field of sport and physical exercise," Sensors, vol. 21, no. 18, Art. no. 5996, 2021, doi: 10.3390/s21185996.

[2] C. Zheng, W. Wu, C. Chen, T. Yang, S. Zhu, J. Shen, N. Kehtarnavaz, and M. Shah, "Deep learning-based human pose estimation: A survey," ACM Computing Surveys, vol. 56, no. 1, Art. no. 11, 2023, doi: 10.1145/3603618.

[3] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, "MediaPipe: A framework for building perception pipelines," arXiv:1906.08172, 2019, doi: 10.48550/arXiv.1906.08172.

[4] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, "BlazePose: On-device real-time body pose tracking," arXiv:2006.10204, 2020, doi: 10.48550/arXiv.2006.10204.

[5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems 33 (NeurIPS 2020), 2020. [Online]. Available: arXiv:2005.11401.

[6] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2023, doi: 10.48550/arXiv.2312.10997.

[7] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," ACM Computing Surveys, vol. 55, no. 12, Art. no. 248, 2023, doi: 10.1145/3571730.

[8] A. Sen, K. Deb, P. K. Dhar, and T. Koshiba, "CricShotClassify: An approach to classifying batting shots from cricket videos using a convolutional neural network and gated recurrent unit," Sensors, vol. 21, no. 8, Art. no. 2846, 2021, doi: 10.3390/s21082846.

[9] H. U. R. Siddiqui, F. Younas, F. Rustam, E. S. Flores, J. B. Ballester, I. de la T. Diez, S. Dudley, and I. Ashraf, "Enhancing cricket performance analysis with human pose estimation and machine learning," Sensors, vol. 23, no. 15, Art. no. 6839, 2023, doi: 10.3390/s23156839.

[10] G. Thomas, R. Gade, T. B. Moeslund, P. Carr, and A. Hilton, "Computer vision for sports: Current applications and research topics," Computer Vision and Image Understanding, vol. 159, pp. 3–18, 2017, doi: 10.1016/j.cviu.2017.04.011.

[11] A. Toshev and C. Szegedy, "DeepPose: Human pose estimation via deep neural networks," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2014, pp. 1653–1660.

[12] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, "OpenPose: Realtime multi-person 2D pose estimation using part affinity fields," IEEE Trans. Pattern Analysis and Machine Intelligence, vol. 43, no. 1, pp. 172–186, 2021, doi: 10.1109/TPAMI.2019.2929257.

[13] Google AI Edge, "Pose landmark detection guide," MediaPipe Solutions documentation. [Online]. Available: https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker (accessed Oct. 2026).

[14] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in Proc. 2020 Conf. Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 6769–6781, doi: 10.18653/v1/2020.emnlp-main.550.

[15] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. 2019 Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP), 2019, pp. 3982–3992, doi: 10.18653/v1/D19-1410.

[16] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," IEEE Trans. Pattern Analysis and Machine Intelligence, vol. 42, no. 4, pp. 824–836, 2020, doi: 10.1109/TPAMI.2018.2889473.

[17] R. Nogueira and K. Cho, "Passage re-ranking with BERT," arXiv:1901.04085, 2019, doi: 10.48550/arXiv.1901.04085.

[18] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, "CPack: Packed resources for general Chinese embeddings," in Proc. 47th Int. ACM SIGIR Conf. Research and Development in Information Retrieval (SIGIR '24), Washington, DC, USA, 2024, doi: 10.1145/3626772.3657878.

[19] Qwen Team (A. Yang et al.), "Qwen3 technical report," arXiv:2505.09388, 2025, doi: 10.48550/arXiv.2505.09388.

[20] K. VanLehn, "The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems," Educational Psychologist, vol. 46, no. 4, pp. 197–221, 2011, doi: 10.1080/00461520.2011.611369.

[21] BAAI, "bge-base-en-v1.5 model card," Hugging Face. [Online]. Available: https://huggingface.co/BAAI/bge-base-en-v1.5 (accessed Oct. 2026).

[22] S. Ramírez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com (accessed Oct. 2026).

[23] M. B. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015, doi: 10.17487/RFC7519.

[24] G. Bradski, "The OpenCV library," Dr. Dobb's Journal of Software Tools, 2000.

[25] Chroma, "Chroma documentation." [Online]. Available: https://docs.trychroma.com (accessed Oct. 2026).

[26] Ollama, "Ollama documentation." [Online]. Available: https://docs.ollama.com (accessed Oct. 2026).

[27] Beanie, "Beanie ODM documentation." [Online]. Available: https://beanie-odm.dev (accessed Oct. 2026).. 248, 2023, doi: 10.1145/3571730.